AUCTION MANAGEMENT PLATFORM: A FULL-STACK WEB-BASED SYSTEM FOR TRANSPARENT REAL-TIME BIDDING

Masuma Fathema and Avinash Yarukala
Department of Computer Science and Engineering, KLH CSE Bowrampet Campus
Under the guidance of Mrs. E. Uma Shankari

Abstract—Traditional auction processes can restrict participation to particular locations and often depend on manual coordination between sellers and bidders. These limitations can create delays in bid tracking, auction closure, payment settlement, and communication. This paper presents the design of an Auction Management Platform intended to support the auction lifecycle through a centralized web application. The proposed system uses React.js for the user interface, Spring Boot and REST APIs for backend services, MySQL for persistent data storage, Spring Security and JSON Web Tokens (JWT) for authentication and role-based authorization, and WebSocket communication for real-time bid updates. The platform is designed around buyer, seller, and administrator roles. Sellers can list items and schedule auctions, while bidders can submit bids subject to minimum-increment validation. The design also includes proxy bidding, automatic winner identification at auction closure, payment-gateway integration, invoice generation, email/SMS notifications, and an administrative dashboard. An Agile, module-based development approach is adopted, covering requirements, system and database design, implementation, integration, testing, and deployment. This paper documents the proposed architecture and workflow; quantitative performance and user-study results are not claimed because verified test measurements were not provided in the project review material.

Keywords—online auction, auction management, real-time bidding, WebSocket, proxy bidding, Spring Boot, React.js, role-based access control

I. INTRODUCTION

Online auctions provide a way for buyers and sellers to exchange goods without being physically present at the same location. However, an effective auction platform must coordinate user access, item listings, auction schedules, bid validation, closing rules, and post-auction transactions. If these activities are handled manually or across disconnected services, users may experience delayed updates, inconsistent records, and uncertainty about the final outcome.

The Auction Management Platform proposed in this project aims to provide a centralized web-based environment for the major stages of an auction. The project review material identifies five key issues: geographical limitations, potential disputes caused by insufficient transparency, inefficient manual coordination, delayed bid tracking, and delayed payment settlement. The system therefore combines user and item management with real-time bidding, auction closure, payment-related workflows, notifications, and administrative monitoring.

The main contributions of this work are: (1) a full-stack architecture using React.js, Spring Boot, REST APIs, and MySQL; (2) real-time bid synchronization through WebSocket communication; (3) validation of minimum bid increments and support for proxy bidding; and (4) a proposed integrated workflow for role-based access, winner identification, payments, invoices, notifications, and administration. The remainder of the paper reviews related work, explains the proposed methodology and architecture, discusses evaluation considerations and limitations, and concludes with future directions.

II. RELATED WORK

Rumpe and Wimmel describe a framework for real-time online auctions and discuss the architectural and technical requirements involved in building a robust web-based auction system [1]. Their work emphasizes the importance of architecture, reuse, and iterative development. These considerations inform the present project's modular design and staged integration approach.

Cai, Chen, and Gong compare proxy and non-proxy online auction settings [2]. Their analysis examines bidding behavior and seller revenue, and reports that proxy settings can outperform non-proxy settings in terms of expected seller revenue under the studied conditions. This work provides a rationale for considering automated bidding, while also indicating that proxy bidding rules need to be designed carefully rather than treated as a universal guarantee of truthful behavior.

Borkar, Dave, and Shyamasundar present SeTiA, a secure multi-auction system, across work on auction mechanisms and system architecture [3], [4]. The system addresses multiple auction mechanisms, confidentiality, auction closure, and secure transactions. This research highlights security and transaction handling as important parts of auction-system design. In contrast, the present project focuses on a contemporary full-stack application integrating WebSocket-based bid updates and role-specific workflows.

The project gap identified from the review material is the need to bring auction scheduling, minimum-increment validation, real-time updates, automated bidding, payment-related processing, and notifications into one platform. The proposed system is intended to address this integration gap. Its effectiveness must be established through implementation and testing rather than inferred from the architecture alone.

III. METHODOLOGY

A. Development Approach

The project adopts an Agile, module-based development approach. The phases specified in the project review are requirement analysis, system design, database design, user management, item and auction management, bidding-engine development, payment and notification integration, module integration, testing, and deployment. Individual modules are to be tested before they are progressively integrated, followed by system-level testing.

B. Technology Stack

The frontend is designed with React.js, HTML5, CSS3, and JavaScript. Axios is used for communication between the user interface and backend APIs. The backend uses Spring Boot to implement application services and REST endpoints. Spring Security and JWT support authentication and role-based authorization, while WebSocket communication is intended to distribute bid updates to connected clients. MySQL stores persistent application records. Git and GitHub are used for version control, and a payment gateway API is planned for online payment processing.

C. User Roles and Functional Modules

The platform defines three principal roles. Buyers can browse available items, join eligible auctions, and submit bids. Sellers can create item listings and schedule auctions. Administrators can monitor platform activity and access reports. Authorization rules

should ensure that each role can perform only the operations permitted to it.

The major functional modules are user authentication and authorization; item listing and auction scheduling; bid submission and validation; proxy bidding; auction closure and winner identification; payment and invoice processing; notification delivery; and administration. The modules share persistent data through the relational database and communicate through backend services.

D. Bidding Workflow

A seller creates an item listing and configures an auction, including its start and end times and the minimum bid increment. An eligible buyer submits a bid through the web interface. The backend validates the auction state, bidder permissions, and bid amount before accepting the bid. A valid bid is persisted and an update is broadcast through WebSocket communication so that connected participants can see the latest state. Proxy bidding, when enabled, uses a bidder-defined maximum amount and the configured increment rules to determine whether an automatic bid should be placed. At auction closure, the system identifies the highest eligible bid according to the auction rules and records the outcome.

E. Data Design

The project review identifies MySQL as the relational database and specifies storage for user details, item information, auction details, bids, and payment or transaction records. A practical schema can represent these as related entities with unique identifiers and foreign-key relationships. Bid records should retain the auction, bidder, amount, and timestamp; auction records should retain scheduling and status information; and payment records should be linked to the relevant auction outcome. The final schema should be validated against the implemented application before deployment.

IV. SYSTEM ARCHITECTURE

The system follows a layered web architecture. The presentation layer is the React.js application through which buyers, sellers, and administrators interact with the platform. The application layer is implemented in Spring Boot and exposes REST APIs for operations such as authentication, listing management, auction scheduling, bid submission, and administration. A WebSocket channel is used for live bid-state updates. The persistence layer uses MySQL to maintain users, items, auctions, bids, and transaction records. External services may be connected for payment processing and email/SMS delivery.

The separation of the presentation, application, real-time communication, persistence, and external-service responsibilities is intended to make the system easier to maintain and extend. REST APIs handle request-response operations, while WebSockets support updates that should reach connected users without repeated manual refreshes. The backend remains responsible for authoritative bid validation; client-side checks alone must not determine whether a bid is accepted.

A. Security and Consistency

Authentication and authorization are planned through Spring Security and JWT. Role checks should be applied to protected operations, including listing creation, bid placement, and administrative actions. Input validation, secure password handling, restricted access to transaction data, and careful management of authentication tokens are necessary safeguards. Bid acceptance should be handled atomically so that concurrent submissions cannot create inconsistent auction states. The server should validate auction timing and minimum increments using the latest persisted bid state.

B. Auction Closure and Transactions

At the configured end time, the auction should transition to a closed state and prevent further bids. The winning bid should be selected according to the implemented auction rules and stored with the final auction status. Payment processing, invoice generation, and notifications should be triggered only after the outcome is recorded. Payment status should be tracked separately from auction status so that a failed or pending payment does not silently alter the recorded bidding result.

V. RESULTS AND EVALUATION PLAN

The supplied project reviews document the problem definition, objectives, technology stack, and intended system design. They do not provide verified implementation screenshots, test-case outcomes, response-time measurements, load-test results, or user-study data. Therefore, this paper reports the proposed design and does not claim numerical performance improvements or completed experimental validation.

After implementation, functional testing should cover registration and login, role permissions, item listing, auction scheduling, valid and invalid bid submissions, minimum-increment enforcement, proxy bidding, auction closure, winner identification, payment-state updates, invoice generation, and notification delivery. Real-time behavior should be tested with multiple clients connected to the same auction. Concurrency tests should submit bids close together to check that the stored highest bid and broadcast updates remain consistent.

Useful quantitative measures include bid-validation success rate, end-to-end update latency, auction-closure correctness, transaction failure rate, and system behavior under increasing concurrent users. These measures should be collected from repeatable test runs and reported with the test environment and sample sizes. Any results table should be populated only after measurements have been obtained.

VI. DISCUSSION

The proposed platform consolidates the main auction lifecycle into a single application and uses real-time communication to reduce dependence on manual refresh and coordination. Role-based access can separate buyer, seller, and administrator responsibilities, while persistent bid records can support traceability. Proxy bidding may improve convenience for users who cannot monitor an auction continuously, but its rules, maximum-bid handling, and interaction with minimum increments must be clearly specified.

The design also has limitations. WebSocket communication does not by itself guarantee bid correctness, fairness, or scalability; those properties depend on server-side validation, concurrency control, deployment configuration, and testing. Payment gateways and email/SMS services introduce external dependencies that may fail or respond slowly. Security and privacy require particular attention because the platform processes account information, bid histories, and transaction records. The current project review material does not establish that all proposed features have been implemented or that the platform has passed performance or security testing.

VII. CONCLUSION AND FUTURE WORK

This paper presents the design of a web-based Auction Management Platform intended to improve the coordination and transparency of online auctions. The proposed stack combines React.js, Spring Boot, REST APIs, WebSocket communication, MySQL, Spring Security, and JWT. The planned functionality includes role-based buyer, seller, and administrator access; item listing and auction scheduling; bid validation; proxy bidding; automatic winner identification; payment integration; invoice generation; notifications; and administrative reporting.

Future work should complete the implementation and evaluate each module through functional, integration, concurrency, security, and load testing. Further extensions may include audit logs, stronger fraud and abuse detection, improved recovery from payment or notification failures, accessibility testing, and deployment monitoring. Quantitative conclusions should be added only after the system has been tested under a documented experimental setup.

ACKNOWLEDGMENT

The authors thank Mrs. E. Uma Shankari for guidance and support during the development of the Auction Management Platform project.

REFERENCES

[1] B. Rumpe and G. Wimmel, “A Framework for Realtime Online Auctions,” arXiv:1409.6592, 2014. [Online]. Available: https://arxiv.org/abs/1409.6592.

[2] G. Cai, Y.-J. Chen, and X. Gong, “Design of online auctions: Proxy versus non-proxy settings,” Decision Support Systems, vol. 52, no. 2, pp. 384–394, 2012, doi: 10.1016/j.dss.2011.09.005.

[3] V. S. Borkar, M. S. Dave, and R. K. Shyamasundar, “Design and Implementation of SeTiA: Secure Multi Auction System I: Auction Mechanisms,” Journal of Intelligent Systems, vol. 14, no. 1, pp. 45–68, 2005, doi: 10.1515/jisys.2005.14.1.45.

[4] V. S. Borkar, M. S. Dave, and R. K. Shyamasundar, “Design and Implementation of SeTiA: Secure Multi Auction System II: Architecture and Implementation Issues,” Journal of Intelligent Systems, vol. 14, no. 1, pp. 69–93, 2005, doi: 10.1515/jisys.2005.14.1.69.